

Microblog Platform (Demo)

A multi-tenant microblog with real-time fan-out timelines, threaded conversations, media uploads, full-text search, and notifications.

Generated January 15, 2026 at 1:00 PM · demo-seed

Executive Summary

The Microblog Platform delivers real-time timelines, threaded conversations, media uploads, full-text search, and DMs. Primary actors are Readers, Authors, Moderators, and Admins working against an Angular 17 SPA, a FastAPI backend, ARQ workers, PostgreSQL 16, Redis 7, S3, and OpenSearch. DDD aggregates, Clean Architecture, and shared Zod-Pydantic schemas enforce boundary discipline.

8 sections

Contents

Executive Summary	Section 1
System Overview	Section 2
Architecture Layers	Section 3
Per-Layer Spec Kit Highlights	Section 4
Domain Deep Dive	Section 5
Key Workflows	Section 6
Architecture Decisions (ADRs)	Section 7
Glossary & Appendix	Section 8

Executive Summary

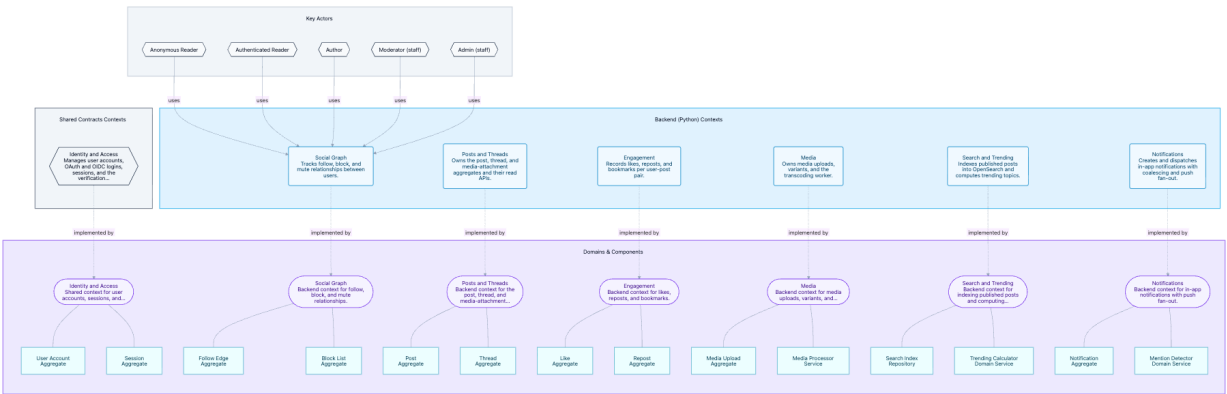
The Microblog Platform replaces a legacy single-server blog with a distributed multi-tenant microblog. Primary actors are Anonymous Readers, Authenticated Readers, Authors, Moderators, and Admins. The headline shape couples an Angular 17 SPA to a Python FastAPI backend, ARQ workers, PostgreSQL 16, Redis 7, S3, and OpenSearch. Key decisions include fan-out on write for sub-200ms timelines, OpenSearch for sub-100ms typeahead, and envelope encryption for DMs at rest. DDD aggregates, Clean Architecture layering, and shared Zod-Pydantic schemas enforce contract discipline across every boundary.

System Overview

The Microblog runtime serves five actor types across a Python 3.11 FastAPI API, ARQ workers, and an Angular 17 SPA. Persistence is PostgreSQL 16, Redis 7, S3, and OpenSearch; CI/CD runs on GitHub Actions and Argo CD.

Anonymous Reader	Browses public timelines without auth
Authenticated Reader	Signs in via OAuth; likes, replies, bookmarks
Author	Composes posts, threads, replies, and media
Moderator (staff)	Triages reports via OIDC; enforces rules
Admin (staff)	Manages users, verification, and platform config
Headline NFR	99.9% availability for read APIs; p95 home feed < 1.5s on warm cache

Figure 1 —



Blueprint — actors to contexts to components

The blueprint traces actor entry through the Identity and Access context, across Posts and Threads, Social Graph, Engagement, Media, Search, and Notifications, into the worker pool. This is the canonical map of how a request becomes a domain event.

Figure 2 —

C4 context — how users reach the system

C4 context shows Readers, Authors, and Staff as primary persons reaching the Microblog. The Microblog integrates with an Identity Provider for OIDC and OAuth, a Push Notification Service, and a Search Indexer.

Figure 3 —

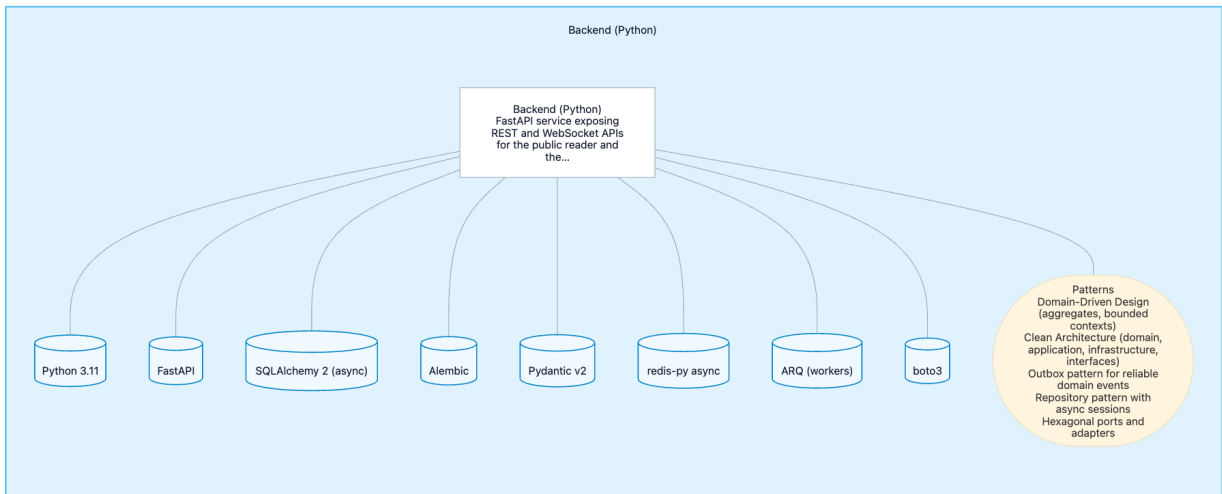
C4 container — runtime topology

C4 container shows the Angular 17 SPA, FastAPI Microblog API, Fan-out, Media, and Notification Workers, plus Primary Store, Cache, and Object Store. Clients use HTTPS and WSS; workers consume Redis Streams.

Architecture Layers

Five layers compose the Microblog stack: backend (Python), backend workers, frontend (Angular), shared contracts, and infrastructure. Each layer owns its tech stack, patterns, and runtime topology.

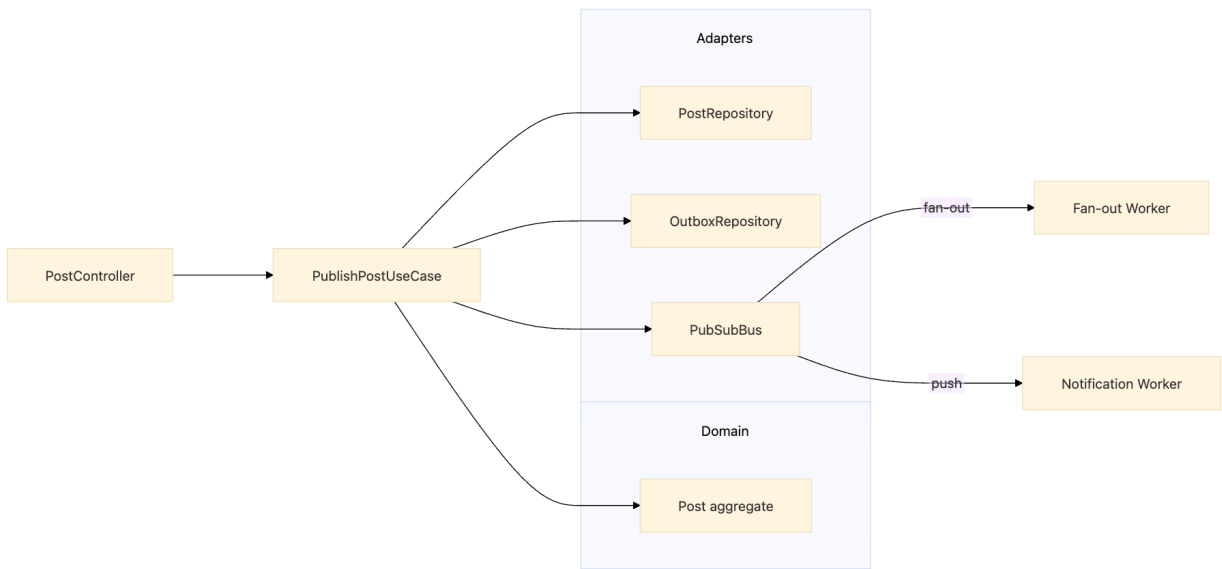
Figure 4 —



Backend — Python 3.11, FastAPI, SQLAlchemy 2 async

name	Backend (Python)
tech stack	Python 3.11, FastAPI, SQLAlchemy 2 (async), Alembic, Pydantic v2, redis-py async, ARQ, boto3
patterns	DDD aggregates per bounded context; Clean Architecture; outbox pattern; repository with async sessions; hexagonal ports and adapters
description	FastAPI service exposing REST and WebSocket APIs for the public reader and staff console, backed by PostgreSQL 16, Redis 7, S3, and OpenSearch.

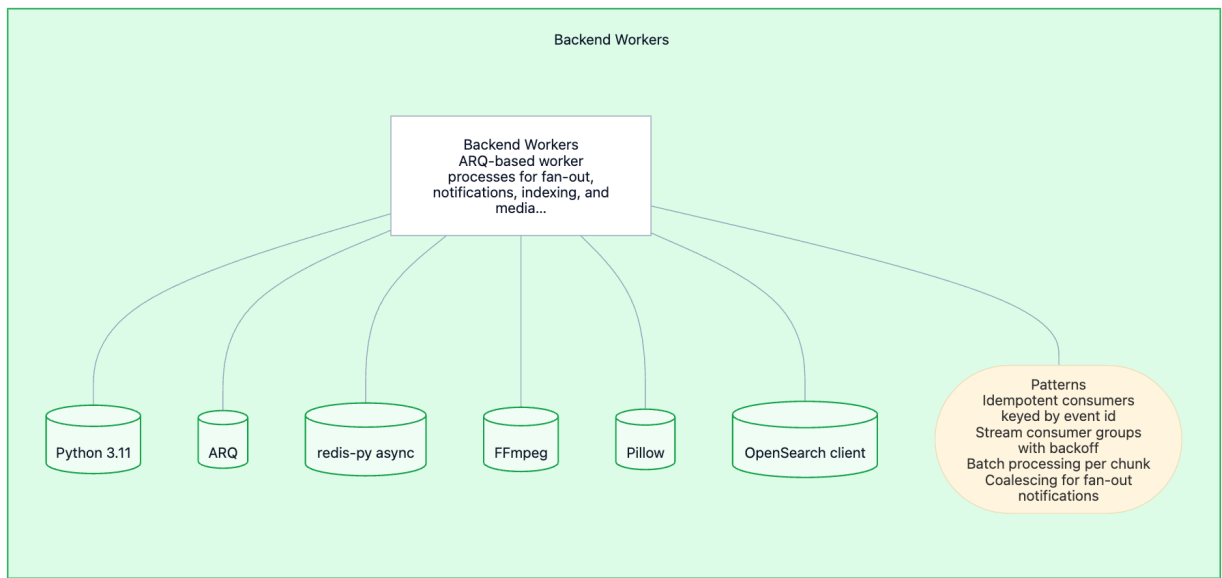
Figure 5 —



Layer overview – FastAPI controller, use case, aggregate, and outbox

PostController drives PublishPostUseCase which mutates the Post aggregate, persists via PostRepository, and writes to OutboxRepository. PubSubBus then fans out to Fan-out and Notification workers.

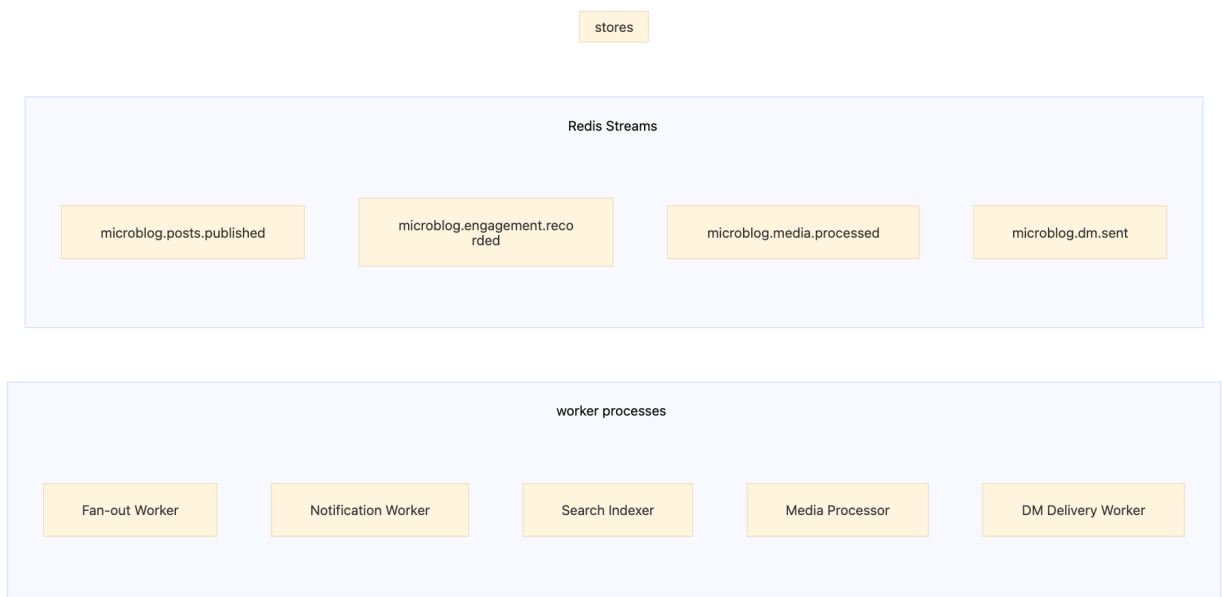
Figure 6 –



Backend workers – Python 3.11, ARQ, redis-py async, FFmpeg

name	Backend Workers
tech stack	Python 3.11, ARQ, redis-py async, FFmpeg, Pillow, OpenSearch client
patterns	Idempotent consumers keyed by event id; stream consumer groups with backoff; batch processing per chunk; coalescing for fan-out notifications
description	ARQ-based worker processes for fan-out, notifications, indexing, and media transcoding, with Redis Streams as the unified event bus.

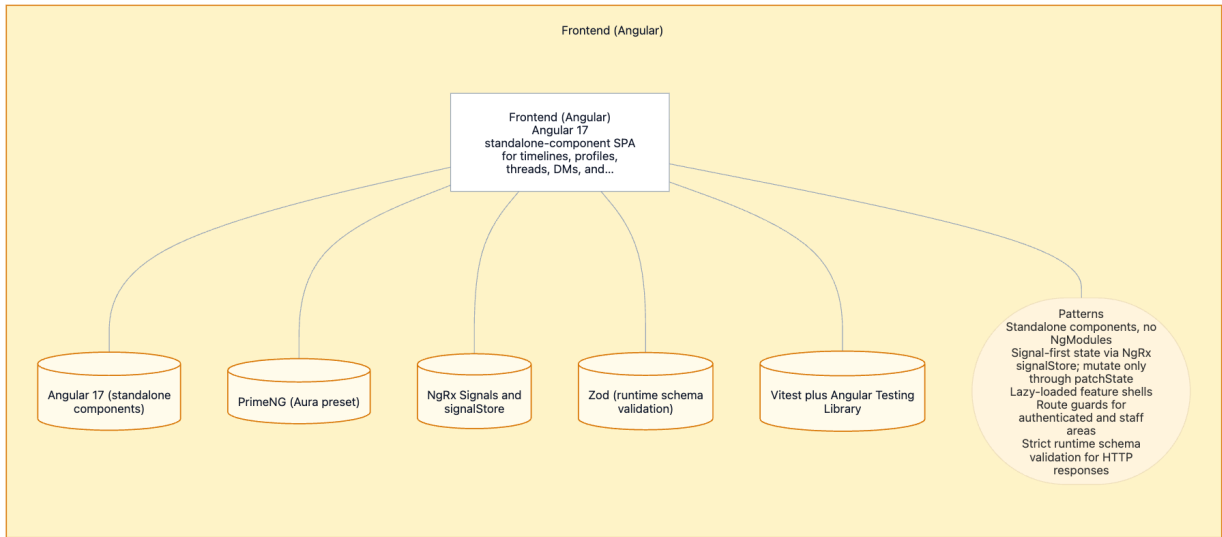
Figure 7 –



Layer overview – five worker kinds consuming dedicated Redis Streams

Fan-out, Notification, Search Indexer, Media Processor, and DM Delivery workers each consume one Redis Stream via XREADGROUP. Workers deduplicate by event id and write back to PostgreSQL, Redis, S3, or OpenSearch.

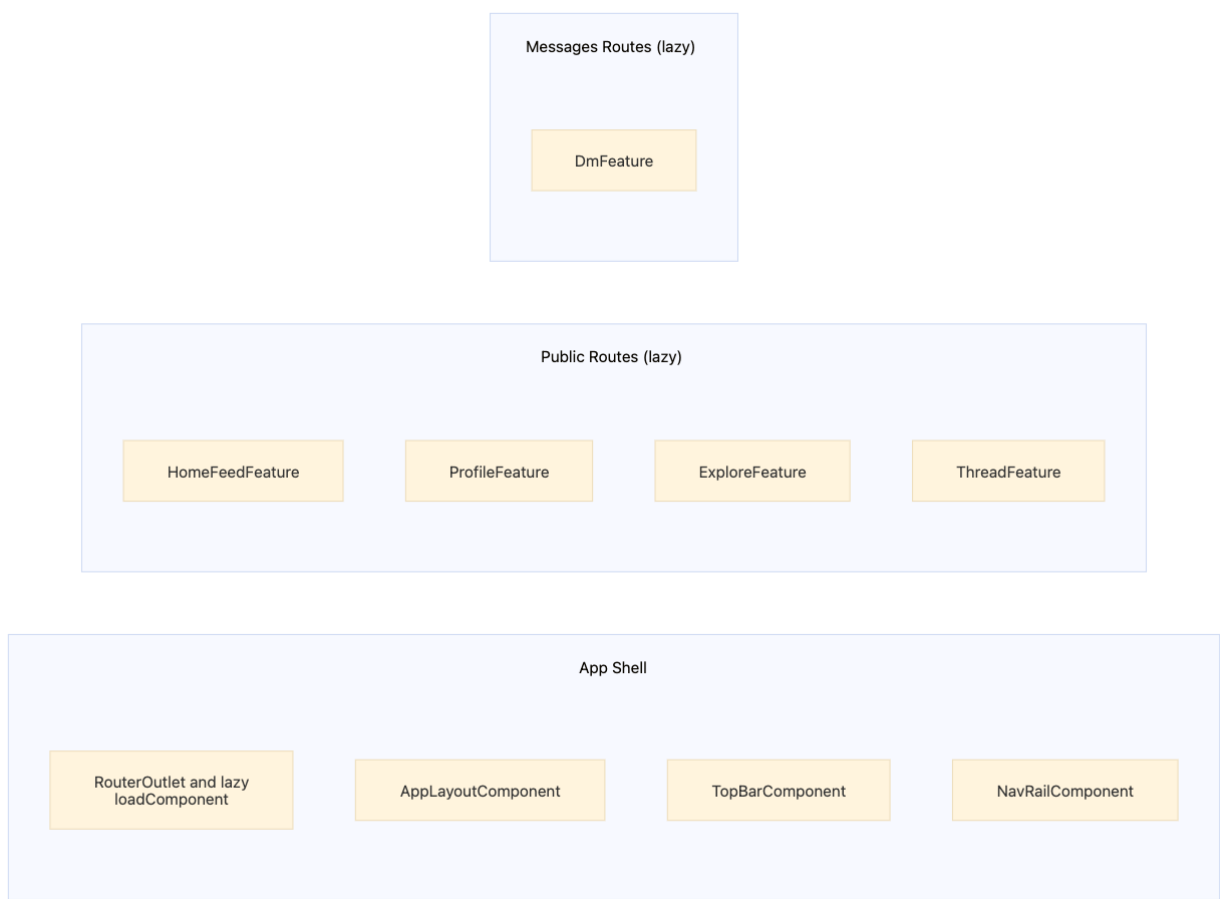
Figure 8 –



Frontend – Angular 17, PrimeNG Aura, NgRx signalStore, Zod

name	Frontend (Angular)
tech stack	Angular 17 standalone components, PrimeNG Aura preset, NgRx Signals and signalStore, Zod, Vitest plus Angular Testing Library
patterns	Standalone components only; signal-first state via NgRx signalStore mutated only via patchState; lazy-loaded feature shells; route guards; strict Zod runtime validation
description	Angular 17 standalone-component SPA for timelines, profiles, threads, DMs, and notifications, with feature shells lazy-loaded by RouterOutlet.

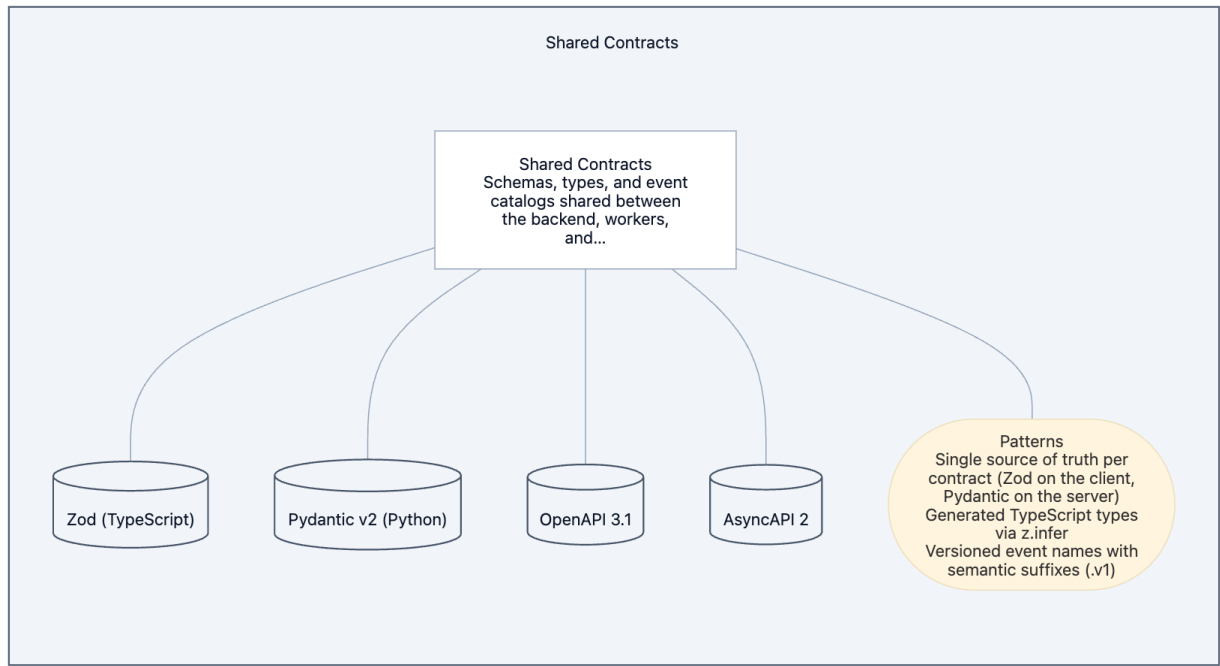
Figure 9 –



Layer overview – app shell, lazy features, and stores

The App Shell hosts RouterOutlet, AppLayoutComponent, TopBarComponent, and NavRailComponent. Public and Messages routes lazy-load feature shells that read from signalStores and call the core api client.

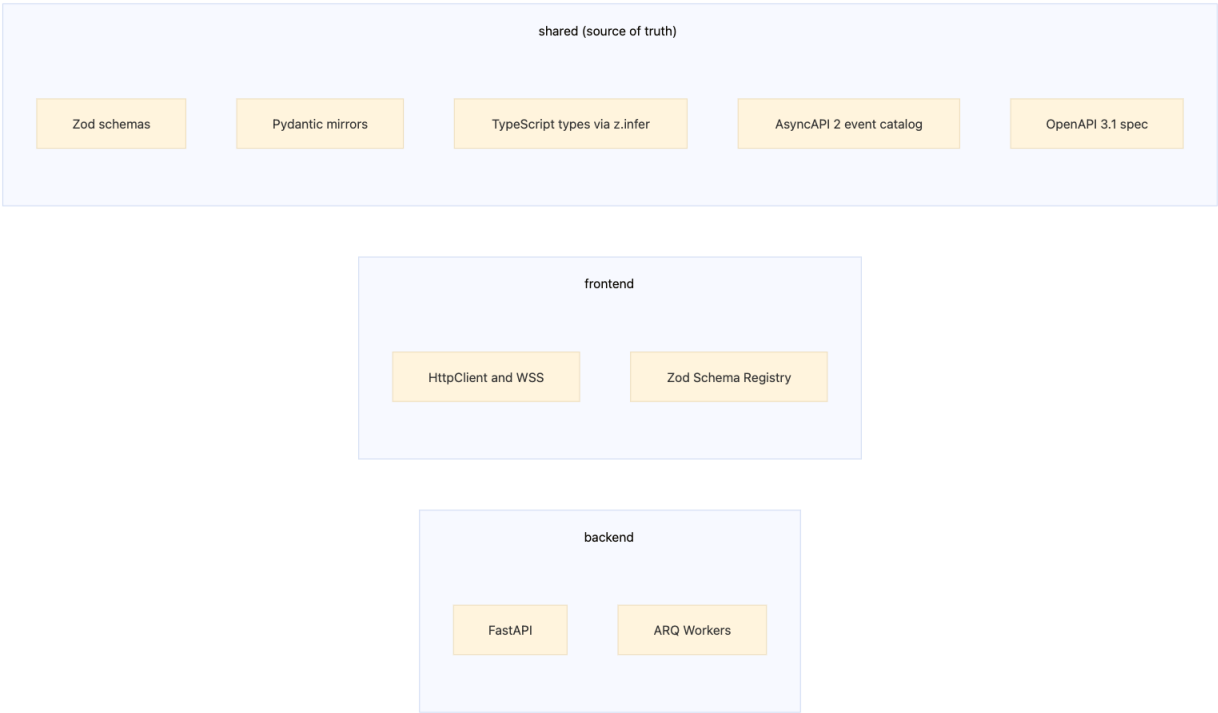
Figure 10 –



Shared – Zod, Pydantic v2, OpenAPI 3.1, AsyncAPI 2

name	Shared Contracts
tech stack	Zod (TypeScript), Pydantic v2 (Python), OpenAPI 3.1, AsyncAPI 2
patterns	Single source of truth per contract; generated TypeScript types via z.infer; versioned event names with .v1 suffix
description	Single source of truth for every API request, response, and domain event crossing the backend, workers, and frontend boundary.

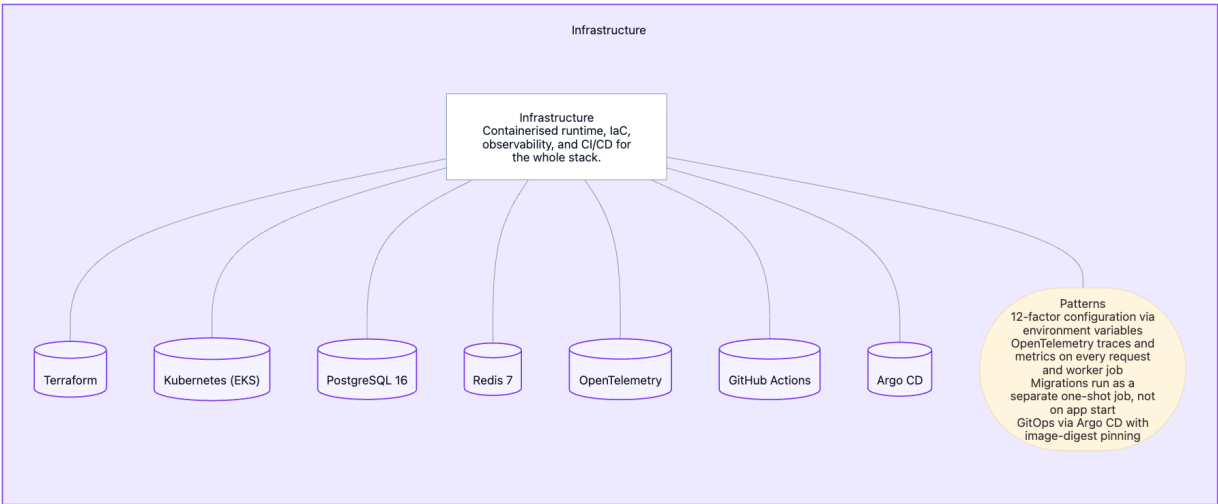
Figure 11 —



Layer overview — Zod root, Pydantic mirror, and codegen pipeline

Backend and frontend both import from a single Zod root schema. Codegen emits Pydantic mirrors, OpenAPI 3.1, AsyncAPI 2, and TypeScript types via z.infer; CI fails on any uncommitted generated artefact.

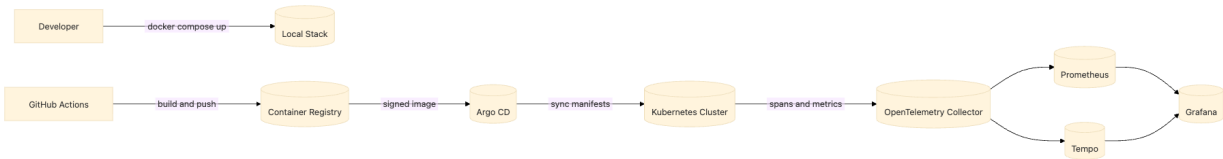
Figure 12 —



Infrastructure — Terraform, Kubernetes EKS, OpenTelemetry, Argo CD

name	Infrastructure
tech stack	Terraform, Kubernetes (EKS), PostgreSQL 16, Redis 7, OpenTelemetry, GitHub Actions, Argo CD
patterns	12-factor configuration via env vars; OpenTelemetry traces and metrics on every request and worker job; migrations as separate one-shot job; GitOps via Argo CD with image-digest pinning
description	Containerised runtime, IaC, observability, and CI/CD. Local dev runs docker compose; production runs on EKS managed by Argo CD with image digests pinned to the commit SHA.

Figure 13 —



Layer overview — CI builds, Argo CD sync, EKS, and OTel fan-out

GitHub Actions builds by digest and pushes to the container registry. Argo CD syncs manifests to EKS, and OpenTelemetry exports traces to Tempo and metrics to Prometheus, both surfaced through Grafana.

Per-Layer Spec Kit Highlights

Five layers expose distinct technicalContext rows and a small set of complexityTracking violations. The matrices below surface the differences side by side; the bullets recap each layer's constitutionCheck rules.

	backend	backend-workers	frontend	infra
storage	PostgreSQL 16 + outbox	Redis Streams; PG/ S3/OS sinks	Browser memory only	GH Actions cache + registry
targetPlatform	Linux + Docker	Linux amd64 + arm64	Chrome 110+, FF 110+, Safari 16+	Linux + Kubernetes EKS
performanceGoals	p95 home 200ms; post 400ms	10k events/min; p95 < 1s	TTI < 3s; LCP < 2s	CI < 12m; deploy < 5m
constraints	Zero framework in domain	Idempotent on event id	No BehaviorSubject	Pin by digest, not tag
scaleScope	1M MAU; 50k posts/min	1k jobs/replica; 6 kinds	1M MAU; 10k DM sessions/day	3 envs; ~80 svcs; ~20TB PG

technicalContext matrix — rows are layer-distinct fields, columns are runtime layers

Performance goals are measurable per request or worker job; constraints are the hard rules each layer must enforce at compile and runtime. Shared Contracts is omitted to fit the four-column matrix cap.

- Backend: strict Pydantic schemas (extra forbid)
- Backend: per-use-case tests (>=2 unit, >=1 integration)
- Backend: zero framework imports in domain layer
- Backend: outbox pattern for every domain event
- Backend: all authorization checks in application layer
- Backend Workers: every job idempotent on event id
- Backend Workers: consume events only, never call API
- Backend Workers: emit OpenTelemetry span per job
- Backend Workers: per-stream DLQ after 5 retries
- Backend Workers: expose /healthz endpoint
- Frontend: strict TypeScript (noImplicitAny, strictNullChecks)
- Frontend: standalone components only (no NgModule)
- Frontend: mutate signalStore only via patchState
- Frontend: Zod-validate every HTTP response
- Frontend: lazy-load every top-level feature
- Shared: Zod is source of truth; TS and Pydantic generated
- Shared: every event declared in AsyncAPI catalog
- Shared: OpenAPI 3.1 regenerated on every CI run
- Shared: breaking changes require .vN suffix
- Shared: schemas imported by both backend and frontend
- Infrastructure: all images pinned by digest, not tag
- Infrastructure: migrations run as separate one-shot job

- Infrastructure: OTel traces and metrics on every request
- Infrastructure: every alert links to a runbook
- Infrastructure: CI fails on uncommitted generated artefacts

	backend	backend-workers	frontend	infra
Outbox table for domain events	used	-	-	-
Repository pattern + Unit of Work	used	-	-	-
Redis Streams consumer groups	-	used	-	-
NgRx signalStore (vs plain service)	-	-	used	-
Zod runtime validation per response	-	-	used	-
Image digests vs mutable tags	-	-	-	used

complexityTracking violations — sparse matrix marking the adopting layer

Shared's Zod-Pydantic schema-parity violation and infra's OTel plus Prometheus plus Grafana stack are also tracked in the plan but omitted here for brevity.

Domain Deep Dive

Seven bounded contexts own the Microblog domain: Identity and Access, Social Graph, Posts and Threads, Engagement, Media, Search and Trending, Notifications. Each owns its aggregates, responsibilities, and emitted events.

- Validate OAuth and OIDC bearer tokens against the configured IdP
- Resolve tokens to UserAccount aggregates with handle and verification state
- Issue and revoke sessions on login, logout, and password reset
- Enforce self-follow rejection on the FollowEdge aggregate
- Track block edges and hide blocked authors from timelines
- Track mute edges without breaking follow relationships

FollowEdge	Directed edge from follower to followee with creation timestamp
Invariant: no self-follow	A user cannot follow themselves
Invariant: blocked target	A user cannot follow a user who has blocked them

- graph.followed.v1 — FollowEdge.follow() ↗ Fan-out Dispatcher
- Enforce the 280-character body limit and the four-media cap
- Track edits and deletes with a bounded history
- Emit post.published.v1 to the outbox on every publish

Post	A short post with body, status, and bounded edit history
Invariant: body length cap	Body length is at most 280 characters
Invariant: no resurrect	Post with status deleted cannot transition to published
Invariant: media readiness	Post with media refs cannot publish until all media are ready

- post.published.v1 — Post.publish() ↗ Fan-out Dispatcher, Search Indexer, Notification Worker
- post.deleted.v1 — Post.delete() ↗ Fan-out Dispatcher, Search Indexer
- Reject duplicate likes and reposts from the same user
- Maintain per-post engagement count snapshots
- Emit engagement.recorded.v1 for downstream consumers
- engagement.recorded.v1 — Like.like()/Repost.repost()/Bookmark.bookmark() ↗ Fan-out, Analytics
- Issue presigned URLs for client uploads
- Track upload status from requested through ready
- Transcode images and videos into CDN-ready variants
- media.processed.v1 — MediaProcessor.processUpload() ↗ Search Indexer
- Bulk-index published posts into OpenSearch with low latency
- Compute trending hashtags and topics on a rolling 24-hour window
- Provide typeahead search at sub-50ms latency
- Create per-user notifications for likes, reposts, mentions, follows, and DMs
- Coalesce notifications within a 60-second window per user

- Dispatch push notifications via the push provider with retries
- `notification.created.v1` — `Notification.create()` ⌘ Fan-out Scheduler

Key Workflows

Five workflows cover the headline user journeys: publishing and fan-out, engagement and notification, upload and processing, search and trending, and report and enforce. Each flow spans at most two domains.

1. Author opens the composer and clicks Post
2. POST api posts persists the post and writes an outbox event
3. PubSubBus publishes post.published.v1 to the fan-out stream
4. Fan-out dispatcher loads followers and writes timeline rows
5. Timeline cache prepends the new post id to each follower cache
6. SPA home feed receives the new post on the next refresh

- posts
- social-graph

1. Reader likes a post from the home feed
2. POST api likes persists the like and writes engagement.recorded.v1
3. Notification worker detects the engagement and creates a notification
4. Fan-out scheduler coalesces notifications and dispatches push
5. The author sees the notification in the panel within 2 seconds

- engagement
- notifications

1. Author attaches an image in the composer
2. Backend issues a presigned URL and stores the upload in pending status
3. Client uploads to the presigned URL and calls markUploaded
4. Media worker transcodes the upload into thumbnail, medium, and full variants
5. Variants are uploaded to the object store and CDN URLs are returned
6. The post publish flow attaches the variants and emits post.published.v1

- media
- posts

1. Fan-out emits post.published.v1 to the search indexer stream
2. Indexing queue buffers posts up to 500 items or 2 seconds
3. Indexing queue flushes a bulk request to OpenSearch
4. Trending calculator applies half-life decay to engagements
5. Explore page serves the top-N trending hashtags from the snapshot

- search
- posts

1. Reader clicks Report on a post and submits a form
2. POST api reports persists the report and emits report.created.v1
3. Staff reviews the report and applies an enforcement action via the BlockList
4. Backend persists the block and emits graph.blocked.v1
5. The reported post is hidden from the reporter timelines

- social-graph
- posts

Architecture Decisions (ADRs)

Six architecture decisions shape the Microblog stack: fan-out on write, PostgreSQL plus Redis, OpenSearch indexing, S3 plus CDN, NgRx signalStore, and Zod-Pydantic schema parity.

Home timelines must render in under 200ms p95 from a warm cache. Fan-out on read would force a join over follow edges on every read.

Use fan-out on write via a Redis stream and a dedicated worker that materialises per-user timeline rows and caches.

- Hot authors (verified accounts) require sharded workers to keep up
- Unfollow triggers dematerialisation via graph.unfollowed.v1
- Read latency drops from multi-second joins to sub-50ms cache reads

We need strong consistency for posts, follows, and engagement with sub-millisecond read latency for hot timelines.

Use PostgreSQL 16 as the system of record and Redis 7 for the hot-path timeline cache and Redis Streams.

- Postgres handles transactional outbox writes for fan-out reliability
- Redis is a cache; cold cache falls back to Postgres reads
- Redis failover may lose up to 5 seconds of cache writes

Typeahead and full-text search must return results in under 100ms p95 across tens of millions of posts.

Index published posts into OpenSearch with a dedicated indexer worker and a buffered bulk request strategy.

- A deleted post is removed from the index within 5 seconds
- Trending snapshots recomputed with a 60-second TTL
- Index failures fall back to a Postgres full-text path

Media uploads include images up to 50MB and videos up to 500MB and must be served with low latency worldwide.

Store raw and transcoded media in S3-compatible object storage and serve variants through a CDN.

- Presigned URLs keep uploads off the API server
- Variant generation is idempotent on upload id
- CDN cache invalidation required when media moderated

We need fast, signal-based state management with a clear audit boundary between reads and writes.

Standalone Angular components only, NgRx signalStore for feature state, mutate only via patchState inside withMethods.

- Components cannot mutate state directly; they call store methods
- Schema validation runs at the API service boundary
- Smaller bundles and clearer feature ownership

Type drift between the Angular SPA and the FastAPI backend has caused production schema_drift errors.

Write every API contract once in Zod (frontend) and mirror it in Pydantic (backend); generate TS types via z.infer and OpenAPI from Zod.

- No hand-written TypeScript request or response types
- Schema changes require a codegen run on CI
- Breaking changes require a new .vN suffix on the event name

Glossary & Appendix

This appendix consolidates ubiquitousLanguage terms from all seven bounded contexts into a single reference. Entries are grouped by primary entity versus supporting concept.

Post	Short post with body, status, and bounded edit history
Thread	Ordered self-reply chain anchored on a root post
FollowEdge	Directed edge from follower to followee with timestamp
Notification	Per-user record describing like, repost, mention, follow, or DM
MediaUpload	Single upload tracked from request through ready variants
SearchIndex	OpenSearch index of published posts with typeahead
UserAccount	Registered user with handle, display name, and verification state
VerificationBadge	Per-user indicator of trust granted by verification-badge service
Session	Server-side record of authenticated client bound to refresh token
BlockList	Per-user list of blocked accounts; blocks suppress both directions
MuteList	Per-user list of muted accounts; mutes hide posts but keep follows
FanoutScheduler	Coalescing dispatcher deciding when to push a notification